# CN LAB

# EXPT – 8

# WIRESHARK SEQ ACK ANALYSIS

24BCE1389

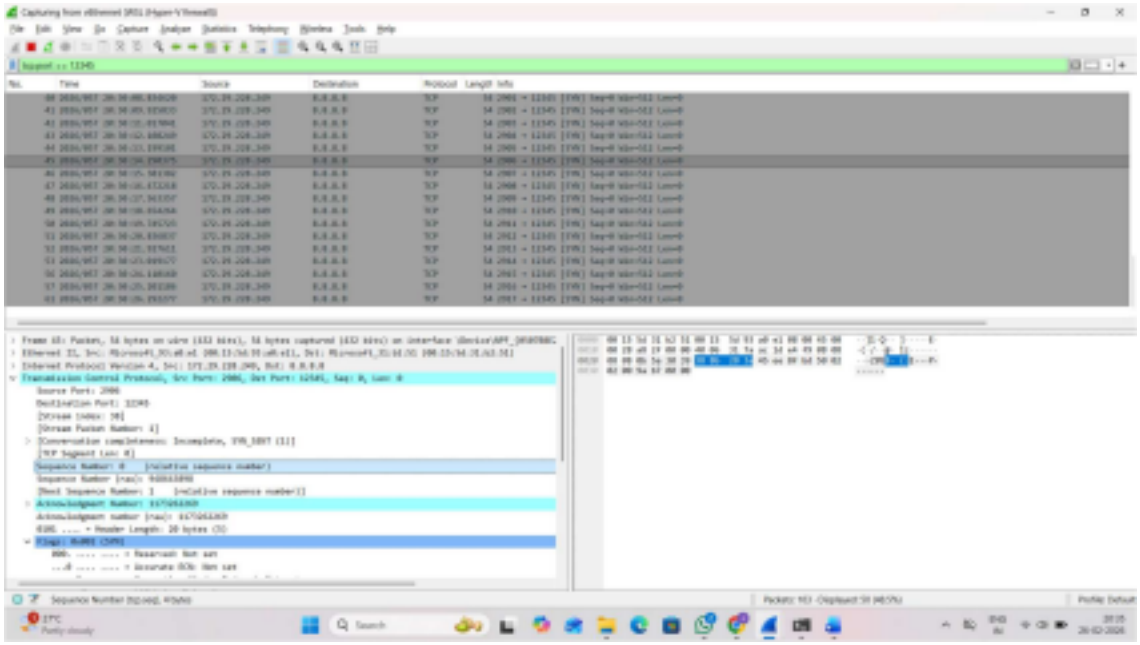
MUTHU VENKATESH M

## TCP SYN Flood:

## Procedure:

1. Open Wireshark on Windows, select the vEthernet (WSL) interface, and start packet capture ,Allow packets to scroll to confirm that traUic capture is active
2. Open the WSL terminal and generate multiple TCP SYN packets using hping3 with a packet count less than 100
3. Return to Wireshark and apply a display filter to isolate SYN packets (tcp.flags.syn == 1 and tcp.flags.ack == 0)
4. Observe a burst of SYN packets appearing in the packet list
5. Verify that packets are directed to the chosen destination port to confirm they originate from hping3
6. Select any SYN packet and expand the Transmission Control Protocol details
7. Observe that only the SYN flag is set, along with sequence number fields and acknowledgement number equal to zero
8. Confirm absence of consistent TCP three-way handshake completion
9. Conclude that repeated SYN packets without handshake completion simulate TCP SYN Flood behavior causing half-open connections.

```
--- 8.8.8.8 hping statistic ---
50 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```



## Explanation:

A TCP SYN Flood attack exploits the TCP connection establishment process by sending a large number of SYN packets without completing the three-way handshake. Normally, a SYN packet initiates a connection, the server responds with a SYN-ACK, and the client
finalizes the session with an ACK. In a SYN Flood, the attacker intentionally avoids sending the final ACK, causing the server to maintain numerous half-open connections. Since the server allocates resources for each request, a high volume of such incomplete sessions can exhaust the connection table, preventing legitimate users from establishing new connections; in Wireshark, this appears as many SYN packets without corresponding handshake completion.

## TCP RST (Reset) Attack:

## Procedure:

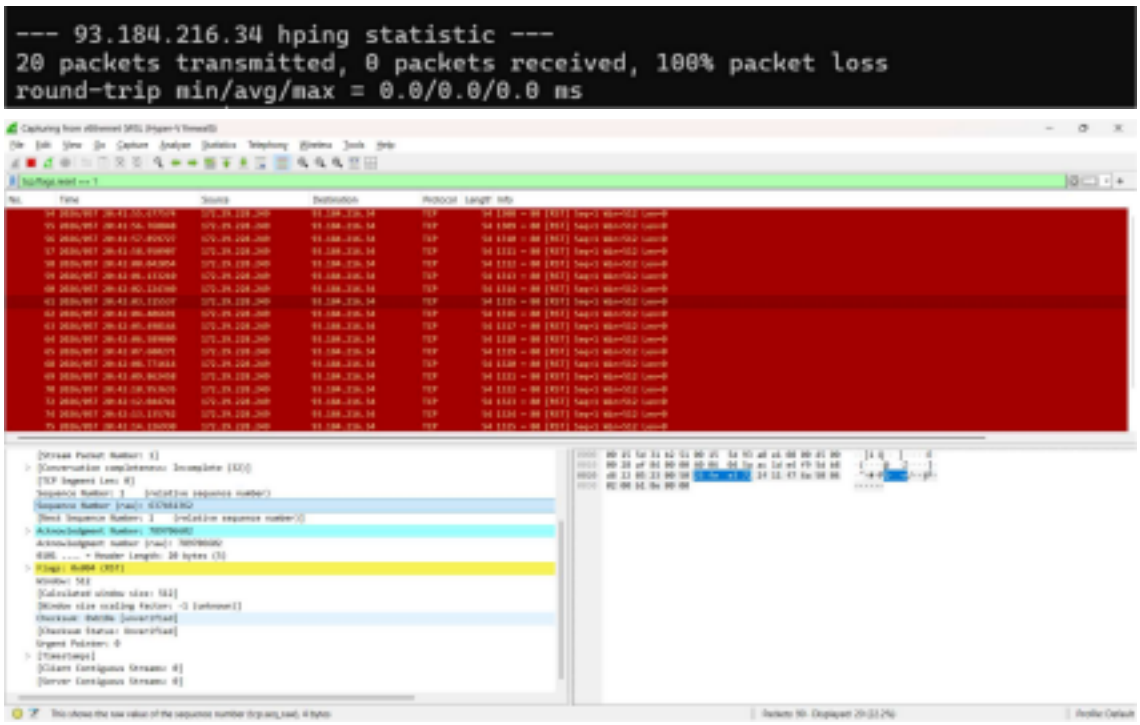
1. Start Wireshark capture on the vEthernet (WSL) interface
2. Ensure packets are visible and capture is active
3. Generate TCP RST packets using hping3 with packet count less

than 100 4. Return to Wireshark while capture is running

5. Apply the display filter:

tcp.flags.reset == 1

6. Observe packets marked [RST] or [RST, ACK] in the packet list
7. Select any RST packet
8. Expand the Transmission Control Protocol section
9. Verify that the Reset (RST) flag is set
10. Observe sequence and acknowledgement number fields
11. Capture screenshot showing RST packets in packet list
12. Capture screenshot showing TCP flags/details
13. Save the capture file
14. Conclude forced connection termination behavior




## Explaination:

A TCP RST (Reset) attack functions by sending forged Reset packets to abruptly terminate an active TCP session. Normally, RST packets are used to immediately close invalid or unexpected connections. In this attack, the injected RST packet appears to originate from a legitimate endpoint, causing the receiving system to instantly drop the connection without following the standard termination

handshake. This results in  sudden session disruption and service interruption, which is commonly observed in  attacks targeting SSH, Telnet, or persistent TCP connections.

## TCP Data Flood:

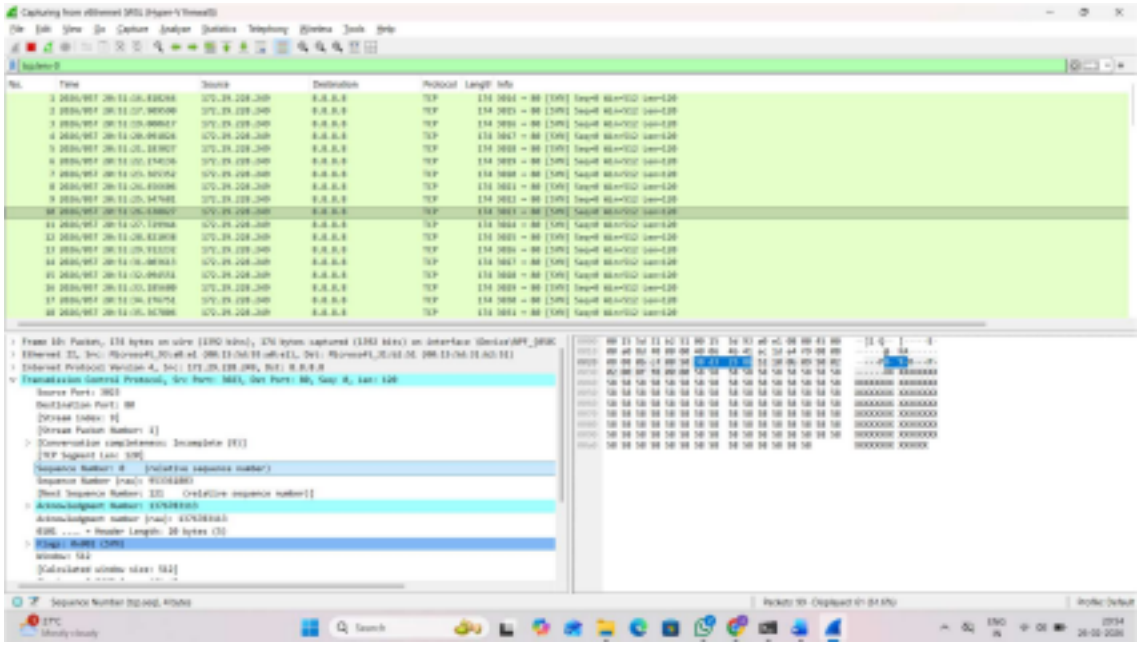
### Procedure:

1. Generate TCP packets with payload data using hping3 while maintaining packet  count below 100
2. Apply the Wireshark display filter: tcp.len > 0
3. Observe TCP packets containing non-zero segment length
4. Verify packets are directed to the selected destination port
5. Select any packet from the packet list
6. Expand the Transmission Control Protocol details
7. Confirm payload presence through non-zero TCP segment length 8.

Conclude repeated payload-bearing packets indicate TCP Data Flood

behavior

```
--- 8.8.8.8 hping statistic ---
50 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```



## Explanation:

A TCP Data Flood attack involves sending a large number of TCP packets containing payload data to overwhelm network bandwidth and processing resources. Unlike SYN Flood attacks, this technique focuses on excessive data transmission rather than connection establishment. In Wireshark, this behavior is identified by observing numerous TCP packets with non-zero segment lengths, indicating continuous payload delivery characteristic of a data flooding scenario.

# TCP Session Hijacking:

## Procedure:

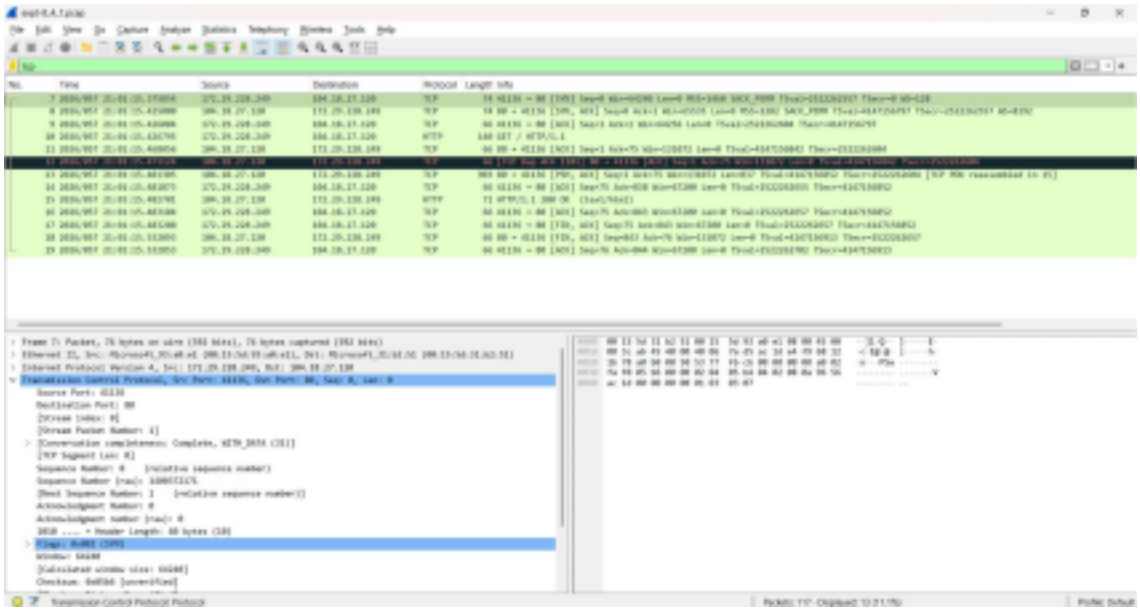
1. Establish a normal TCP connection between two systems (client and server) 2. Observe the active TCP session in Wireshark
3. Identify the sequence and acknowledgement numbers of the ongoing communication
4. Generate crafted TCP packets attempting to mimic the session parameters 5. Apply a Wireshark filter to isolate packets belonging to the selected TCP stream 6. Analyze packets for irregularities in sequence

number progression 7. Observe any unexpected packets appearing within the established session

8. Conclude that packets with valid session parameters but abnormal behavior  indicate TCP Session Hijacking simulation

```
tial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5e
m}div{opacity:0.8}a:link,a:visited{color:348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use
 in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/
example">Learn more</a></p></div></body></html>
```

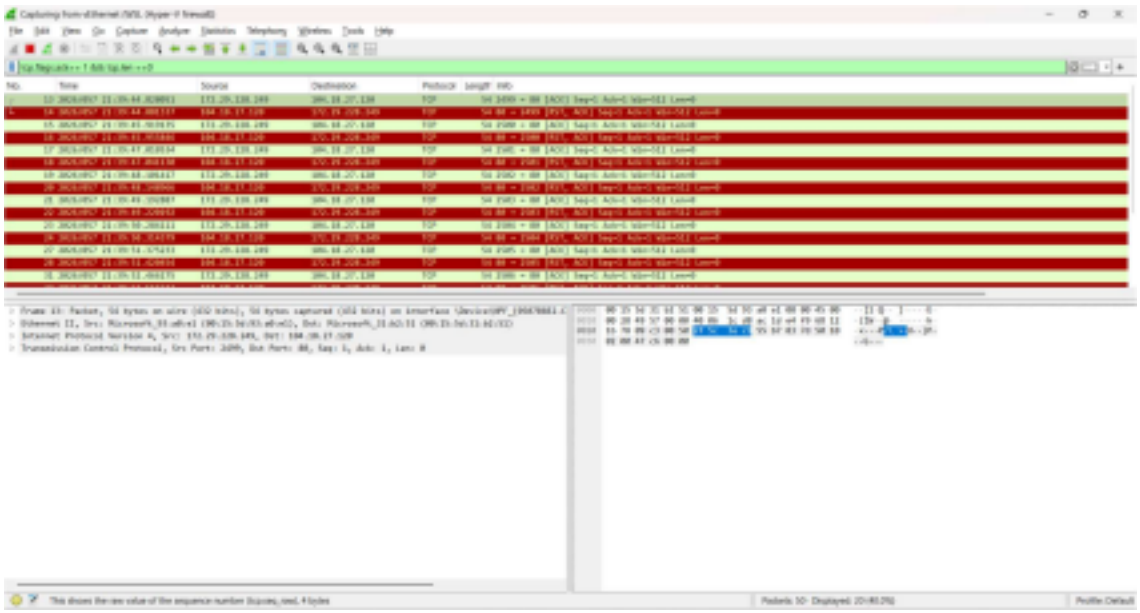

3 – way handshake:



TCP STREAM:

```
HPING example.com (eth0 104.18.27.120): A set, 40 headers + 0 data bytes
len=40 ip=104.18.27.120 ttl=249 DF id=42170 sport=80 flags=RA seq=0 win=512 rtt=55.4 ms
len=40 ip=104.18.27.120 ttl=249 DF id=44218 sport=80 flags=RA seq=1 win=512 rtt=39.0 ms
len=40 ip=104.18.27.120 ttl=249 DF id=46522 sport=80 flags=RA seq=2 win=512 rtt=30.8 ms
len=40 ip=104.18.27.120 ttl=249 DF id=49594 sport=80 flags=RA seq=3 win=512 rtt=46.5 ms
len=40 ip=104.18.27.120 ttl=249 DF id=55226 sport=80 flags=RA seq=4 win=512 rtt=30.3 ms
len=40 ip=104.18.27.120 ttl=249 DF id=57530 sport=80 flags=RA seq=5 win=512 rtt=29.9 ms
len=40 ip=104.18.27.120 ttl=249 DF id=59866 sport=80 flags=RA seq=6 win=512 rtt=45.8 ms
len=40 ip=104.18.27.120 ttl=249 DF id=63162 sport=80 flags=RA seq=7 win=512 rtt=53.4 ms
len=40 ip=104.18.27.120 ttl=249 DF id=65210 sport=80 flags=RA seq=8 win=512 rtt=45.2 ms
len=40 ip=104.18.27.120 ttl=249 DF id=1211 sport=80 flags=RA seq=9 win=512 rtt=36.9 ms

--- example.com hping statistic ---
10 packets transmitted, 10 packets received, 0% packet loss
round-trip min/avg/max = 29.9/41.3/55.4 ms
```



EXPLANATION:

In this experiment, a normal TCP session was first generated to observe standard connection behavior, including proper sequence and acknowledgement number progression. Additional packets were then crafted using hping3 with various TCP flags such as ACK, PSH-ACK, and RST to simulate traUic that resembles session manipulation attempts. These packets do not belong to the legitimate TCP stream and therefore appear as irregular or standalone transmissions in Wireshark. This demonstrates how TCP Session Hijacking conceptually works, where forged packets attempt to mimic valid session characteristics; analysis of TCP flags, payload presence, and Seq/Ack behavior highlights how abnormal packets can indicate potential injection or disruption attempts within an established TCP communication.

## FIN-ACK Flood:

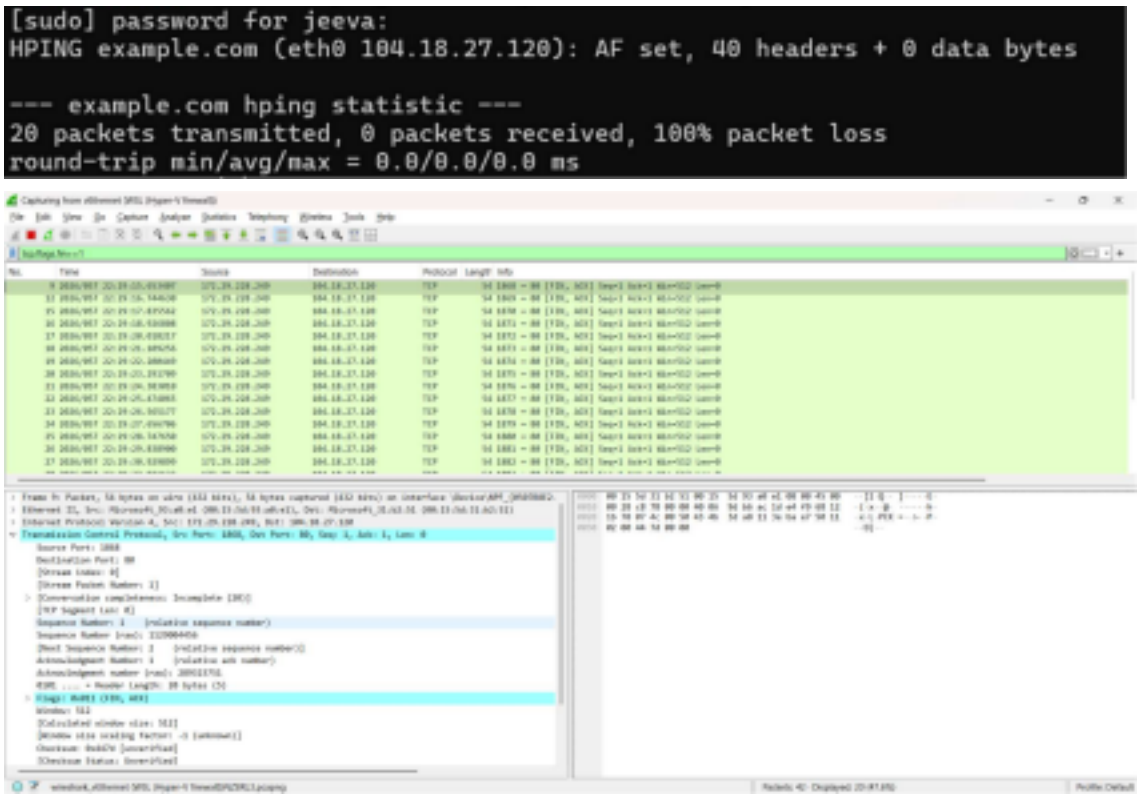
## Procedure

1. Generate FIN-ACK packets using hping3 while keeping packet count below 100
2. Return to Wireshark and apply the display filter:

tcp.flags.fin == 1

3. Observe packets marked [FIN, ACK] in the packet list
4. Verify packets are directed to the selected destination port
5. Select any FIN-ACK packet
6. Expand Transmission Control Protocol details
7. Confirm FIN and ACK flags are set
8. Conclude repeated FIN-ACK packets simulate FIN-ACK Flood

behavior FIN – ACK Generation:




## Explanation:

In this experiment, FIN-ACK packets were generated using hping3 to simulate abnormal TCP termination behavior. These packets mimic legitimate connection closure signals by setting both the FIN and ACK flags. When observed in Wireshark, multiple FIN-ACK packets appear without corresponding normal session context, indicating repeated termination attempts. This demonstrates the concept of a FIN-ACK Flood, where excessive termination requests force the target system to process unnecessary
connection closures, potentially consuming system resources and degrading

network performance.

# TCP Reflection Attacks:

## Procedure

1. Generate TCP packets directed toward a reachable server using hping3
2. Return to Wireshark and apply display filter:
   tcp
3. Observe outgoing TCP request packets
4. Identify corresponding response packets from the server
5. Compare source and destination IP addresses
6. Analyze packet direction and traUic flow patterns
7. Observe how responses are triggered by incoming requests
8. Conclude reflection behavior based on response generation

Generation:

```
HPING example.com (eth0 104.18.27.120): S set, 40 headers + 0 data bytes
len=44 ip=104.18.27.120 ttl=51 DF id=8923 sport=80 flags=SA seq=0 win=65535 rtt=47.8 ms
len=44 ip=104.18.27.120 ttl=51 DF id=11483 sport=80 flags=SA seq=1 win=65535 rtt=47.7 ms
len=44 ip=104.18.27.120 ttl=51 DF id=14299 sport=80 flags=SA seq=2 win=65535 rtt=47.5 ms
len=44 ip=104.18.27.120 ttl=51 DF id=15067 sport=80 flags=SA seq=3 win=65535 rtt=39.3 ms
len=44 ip=104.18.27.120 ttl=51 DF id=15323 sport=80 flags=SA seq=4 win=65535 rtt=47.0 ms
len=44 ip=104.18.27.120 ttl=51 DF id=15579 sport=80 flags=SA seq=5 win=65535 rtt=46.8 ms
len=44 ip=104.18.27.120 ttl=51 DF id=16683 sport=80 flags=SA seq=6 win=65535 rtt=46.7 ms
len=44 ip=104.18.27.120 ttl=51 DF id=17883 sport=80 flags=SA seq=7 win=65535 rtt=46.5 ms
len=44 ip=104.18.27.120 ttl=51 DF id=18139 sport=80 flags=SA seq=8 win=65535 rtt=46.3 ms
len=44 ip=104.18.27.120 ttl=51 DF id=18395 sport=80 flags=SA seq=9 win=65535 rtt=46.2 ms
len=44 ip=104.18.27.120 ttl=51 DF id=19163 sport=80 flags=SA seq=10 win=65535 rtt=46.1 ms
```

```
len=44 ip=104.18.27.120 ttl=51 DF id=19419 sport=80 flags=SA seq=11 win=65535 rtt=45.9 ms
len=44 ip=104.18.27.120 ttl=51 DF id=19931 sport=80 flags=SA seq=12 win=65535 rtt=61.9 ms
len=44 ip=104.18.27.120 ttl=51 DF id=20187 sport=80 flags=SA seq=13 win=65535 rtt=45.7 ms
len=44 ip=104.18.27.120 ttl=51 DF id=21723 sport=80 flags=SA seq=14 win=65535 rtt=37.6 ms
len=44 ip=104.18.27.120 ttl=51 DF id=24539 sport=80 flags=SA seq=15 win=65535 rtt=37.4 ms
len=44 ip=104.18.27.120 ttl=51 DF id=25051 sport=80 flags=SA seq=16 win=65535 rtt=45.2 ms
len=44 ip=104.18.27.120 ttl=51 DF id=25563 sport=80 flags=SA seq=17 win=65535 rtt=52.9 ms
len=44 ip=104.18.27.120 ttl=51 DF id=25819 sport=80 flags=SA seq=18 win=65535 rtt=44.9 ms
len=44 ip=104.18.27.120 ttl=51 DF id=28379 sport=80 flags=SA seq=19 win=65535 rtt=52.8 ms

--- example.com hping statistic ---
20 packets transmitted, 20 packets received, 0% packet loss
round-trip min/avg/max = 37.4/46.6/61.9 ms
```